ADVANCED PROGRAMMING OOPS PROJECT REPORT

Project Title:Random Password Generator using Java OOP

Submitted By:
Name: Rupam Kumar Pal
Roll Number: 20

Course:B.Tech CSE (IOTCSBT)

Instructor Name: Sagatam Basu,Soumadip Biswas And Susovan Jana

Semester:4th Semester

2. PROBLEM STATEMENT

2.1 Introduction

Passwords are one of the most important components of digital security. However, many users still create weak, short, or predictable passwords that can be easily guessed or cracked. Such weak passwords create a serious security risk for personal accounts, email services, social media, and other digital systems.

2.2 Problem Definition

The problem addressed in this project is the generation of secure and customizable passwords. Users often do not know how to create strong passwords manually, especially when different websites require combinations of uppercase letters, lowercase letters, numbers, and symbols.

2.3 Proposed Solution

To solve this problem, a Java-based Random Password Generator application was developed. The application allows users to:

- choose password length
- include uppercase letters
- include lowercase letters
- include numbers
- include symbols
- generate stronger passwords using secure mode
- copy the generated password
- save the password to a file

This project demonstrates how Java OOP concepts can be applied to solve a practical security-related problem.

3. SYSTEM DESIGN

3.1 Overview

The project is designed as a modular Java desktop application using Java Swing for the graphical user interface and multiple interacting classes for the core logic.

The project is divided into three logical packages:

- core → password generation logic
- ui → user interface
- util → utility / file handling

This modular structure improves code readability, maintainability, and scalability.

3.2 Package Structure

The package structure of the project is as follows:
src/com/passwordgen/core
src/com/passwordgen/ui
src/com/passwordgen/util

3.3 Major Classes

3.3.1 Generator

This is an interface that provides abstraction for password generation.

3.3.2 PasswordGenerator

This class contains the base logic for generating random passwords.

3.3.3 SecurePasswordGenerator

This class extends PasswordGenerator and generates stronger passwords using inheritance and polymorphism.

3.3.4 CharacterPool

This class manages character sets such as uppercase letters, lowercase letters, numbers, and symbols.

3.3.5 MainUI

This class handles the graphical user interface and user interaction using Java Swing.

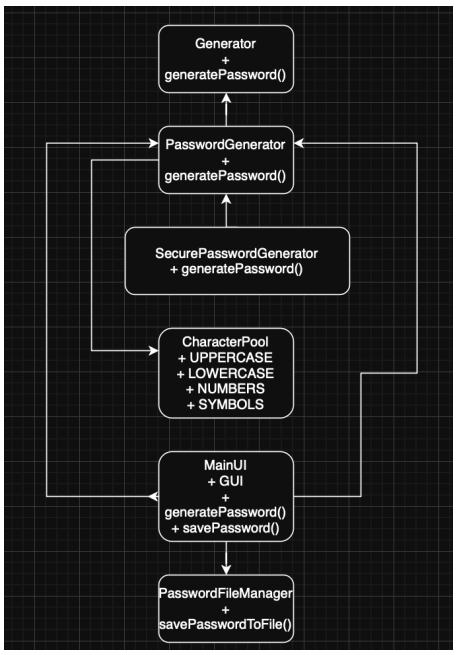
3.3.6 PasswordFileManager

This class is responsible for saving generated passwords to a file.

3.4 Interaction Flow

The high-level interaction flow of the system is:
User
↓
MainUI
↓
PasswordGenerator / SecurePasswordGenerator
↓
CharacterPool
↓
Generated Password
↓
PasswordFileManager

3.5 UML Class Diagram



4. OOP CONCEPTS DEMONSTRATED

This project demonstrates several important Object-Oriented Programming concepts.

4.1 Abstraction

Abstraction is implemented using the Generator interface.
The interface defines the password generation behavior without exposing implementation details.

Code Snippet
```
public interface Generator {
    String generatePassword(int length);
}
```
Explanation

This interface provides a common abstraction for password generation. Different classes can implement or extend this behavior.

4.2 Encapsulation

Encapsulation is used in the CharacterPool class to manage character data and random generation logic in a controlled manner.

Code Snippet
```
public class CharacterPool {
    public static final String UPPERCASE = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    public static final String LOWERCASE = "abcdefghijklmnopqrstuvwxyz";
    public static final String NUMBERS = "0123456789";
    public static final String SYMBOLS = "!@#$%^&*()-_=+<>?";
}
```

Explanation

The character sets are organized inside a separate class so that they can be reused safely and efficiently.

4.3 Inheritance

Inheritance is demonstrated by the SecurePasswordGenerator class extending PasswordGenerator.

Code Snippet
```
public class SecurePasswordGenerator extends PasswordGenerator {
    // secure password generation logic
}
```

Explanation

This allows the secure generator to reuse the base functionality of the parent class while extending its behavior.

4.4 Polymorphism

Polymorphism is used by allowing a common reference to work with different password generation behaviors.

Code Snippet
```
Generator generator = new SecurePasswordGenerator();
```
Explanation

The same interface/reference can be used for different implementations, improving flexibility and scalability.

4.5 Exception Handling

Exception handling is used to validate user input and prevent runtime errors.

Code Snippet
```
try {
    int length = Integer.parseInt(lengthField.getText());
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this, "Please enter a valid number.");
}
```
Explanation

This ensures that invalid user input does not crash the program and provides a user-friendly error message.

4.6 Collections

Collections are demonstrated using List<Character> inside the CharacterPool class.

Code Snippet
```
List<Character> charList = new ArrayList<>();
```
Explanation

Collections are useful for managing character data dynamically during password generation.

4.7 File Handling

File handling is used to save generated passwords into a text file.

Code Snippet
```
FileWriter writer = new FileWriter("saved_passwords.txt", true);
writer.write(password + "\n");
writer.close();
```
Explanation

This feature allows the generated password to be stored for later use.

5. IMPLEMENTATION HIGHLIGHTS

5.1 Java Swing User Interface

The project uses Java Swing to provide a graphical interface.
This makes the application easy to use for non-technical users.

Features in UI:

- password length input
- checkboxes for character types
- secure mode
- generate button
- copy button
- save button

5.2 Secure Mode Design

One important design decision was the use of a separate SecurePasswordGenerator class.
This allowed stronger password generation while maintaining modular code structure.

5.3 Save to File Feature

The application allows users to save generated passwords into a file.
This improves usability and demonstrates Java file handling.

5.4 Clipboard Copy Support

The project also supports copying the generated password to the clipboard, which improves user convenience.

6. TESTING & ERROR HANDLING

6.1 Test Cases Considered

Test Case	Input	Expected Result
Normal generation	Length = 8	Password generated
All options selected	Upper + Lower + Num + Symbols	Strong mixed password
Secure mode enabled	Length = 12	Stronger password generated
Invalid input	abc	Error message shown
Very short secure input	3	Validation / warning

6.2 Edge Cases Handled

The following edge cases were considered:

- non-numeric password length
- empty or invalid input
- too short password length
- secure mode with invalid length
- save operation handling

6.3 Failure Scenarios

Failure scenarios were handled using:

- try-catch blocks
- validation messages
- safe user prompts

This improves the robustness of the application.

7. GIT WORKFLOW

7.1 Version Control Usage

Git and GitHub Classroom were used for project submission and version control.
The project was committed incrementally during development to reflect progress.

7.2 Development Progression

The Git history shows the project development stages, including:

- initial project setup
- password generation logic
- Swing UI implementation
- save feature
- secure mode
- README completion
- slide submission

7.3 Commit History Screenshot

[Insert screenshot of Git commit history here]

(Take screenshot of git log --oneline --graph or GitHub commits page and insert here)

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

This project successfully demonstrates a small but well-structured Java OOP application.
It solves a practical problem by generating secure and customizable passwords.
The project also demonstrates proper use of abstraction, encapsulation, inheritance, polymorphism, exception handling, collections, and file handling.

The modular structure and graphical interface make the application easy to use and defend in an academic OOP setting.

8.2 Future Scope

Possible future improvements include:

- password strength meter
- password encryption
- password history management
- export to CSV or database
- theme customization
- advanced password policies

9. APPENDIX

9.1 Initial Project Proposal

Proposed Title - Secure Random Password Generator

Problem Statement :

Weak passwords are one of the most common causes of security breaches in online systems. Many users create simple passwords that are easy to guess, such as names or predictable number patterns. A secure password generator can help users create strong passwords using a combination of uppercase letters, lowercase letters, numbers, and special characters. This project will develop a Java-based Random Password Generator that allows users to customize password length and character types while ensuring strong security practices. The system will also validate user input and handle invalid values using exception handling. A cryptographically secure random number generator will be used to ensure that the generated passwords are unpredictable and secure.

Target Users:

General Internet Users
Students or Workers
Developers/Administrative System Users

Core Features:

A.Generate Strong random passwords

B.User - Defined password length

C.Option to Include uppercase letters, lowercase letter, numbers, and symbols

D.Input validation for incorrect values

E.Use of secure random generation

F.Simple command-line interface

OOP Concepts to be Used:

Abstraction
Password generation logic will be abstracted through an **abstract class or interface** called `PasswordPolicy`, which defines rules for password creation.

Inheritance
Different password policies such as **BasicPasswordPolicy** and **StrongPasswordPolicy** will inherit from the base `PasswordPolicy` class.

Polymorphism
The `generatePassword()` method will be overridden in different policy classes, allowing the program to generate passwords according to different security rules.

Exception Handling
Invalid inputs such as negative password length or unsupported options will be handled using exceptions like `IllegalArgumentException`.

Collections / Threads
Java Collections (such as `ArrayList` or `List`) will store the character pools used in password generation. Threads are not required for this project because password generation is a lightweight operation.

Proposed Architecture Description:

The system will follow a modular object-oriented architecture consisting of several classes. A Main class will handle user interaction and input validation. A PasswordGenerator class will coordinate password creation using a selected PasswordPolicy. The abstract class PasswordPolicy defines the rules for generating passwords, and concrete classes such as BasicPasswordPolicyand StrongPasswordPolicy implement these rules using inheritance and polymorphism. Character sets (letters, numbers, symbols) will be stored in collections inside a utility class such as CharacterPool. The generator will use java.security.SecureRandom to ensure cryptographically strong randomness while exception handling ensures reliable input validation.

UML Class Structure (Simple Diagram):

```
Main
 |
 | uses
 v
PasswordGenerator
 |
 | uses
 v
PasswordPolicy (abstract)
     ^
     |
 -------------------------
 |                       |
BasicPasswordPolicy   StrongPasswordPolicy

CharacterPool (utility class)
```

UML Class Overview:
The Main class interacts with the user and sends requests to PasswordGenerator. The generator uses an abstract PasswordPolicyclass, which is implemented by different policy types like BasicPasswordPolicy and StrongPasswordPolicy. Character sets are managed in the CharacterPool utility class.

9.2 Approval / Review Mail

S

soumadip.biswas@ieminternational.ai

To: Rupam Kumar Pal

Reply | Reply all | Forward

Tue 10-03-2026 15:44

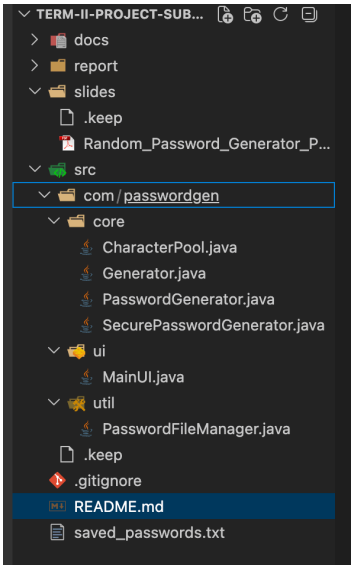
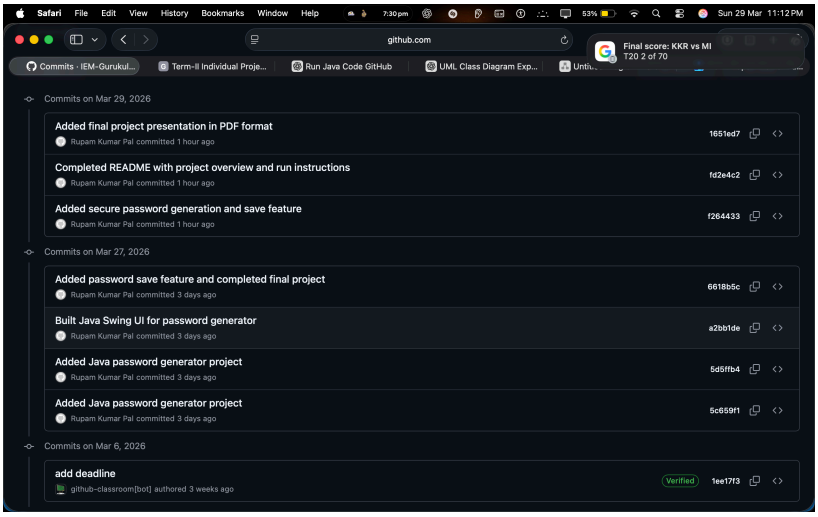
Dear Rupam Kumar Pal,

Project: Random Password Generator
Decision: MAJOR_REVISION
Score: 45

Strengths:
The proposal clearly defines a relevant problem (password insecurity) and proposes a practical solution. The target audience is well-identified, and the core features are logical and address the problem effectively. The intent to use Collections for character sets is a good application of an OOP concept.

Improvements:
1. **Critical Issue: Input Validation:** The statement 'No validation of input will be performed' is fundamentally problematic for a Java OOP project, especially one that lists 'Exception Handling' as a core concept. Robust applications require proper input validation, and students should learn to implement this using exception handling or other appropriate mechanisms. This needs to be completely revised.
2. **Justify OOP Concepts:** The inclusion of 'Inheritance', 'Polymorphism', and 'Threads' in a simple two-class architecture (Main and PasswordGenerator) seems forced and unnatural. These concepts should be applied where they genuinely simplify design, improve extensibility, or solve a specific problem. The current architecture does not provide clear scenarios for their natural application without artificial over-engineering. Consider redesigning to naturally incorporate these (e.g., different password policy types using inheritance/polymorphism, or a more complex UI that would benefit from threading, if applicable).
3. **Architecture Detail:** Expand on the architecture beyond two classes. How would Abstraction be specifically realized? Clarify how 'character sets will be created in a Collection outside of either Class' in Java (e.g., static final fields in a dedicated constants class, or an enum for character types). The current description is too sparse to effectively evaluate the application of the listed OOP principles.
4. **Security Best Practices:** Briefly mention the use of cryptographically strong random number generators (e.g., `java.security.SecureRandom`) to ensure the generated passwords are truly random and secure, rather than relying on less secure `java.util.Random`.

Regards,
Course Instructor

Reply | Forward

9.3 Additional Screenshots